Dasar Go untuk Web

Pemrograman Web • 15 slides • Generated on 11 October 2026, 22:42

Sintaks Go dasar untuk pemrograman web

Dasar Go untuk Web

- **Materi:**
  - **Sintaks dasar**: variabel, tipe data, fungsi
  - **Tipe bentukan**: `struct`, `slice`, `map`
  - **Pointer**: konsep & kapan dipakai
  - **Error handling**: pola `if err != nil`
  - **Package & module**: `go mod init`, `go get`

Anatomi Program Go Minimal

Slide 2

- Program Go tersimpan dalam file berekstensi `.go`
- Syarat pembuatan aplikasi Go yang bisa dijalankan (executable) adalah penggunaan package "main" dan ada fungsi "main" pada salah satu file berekstensi `.go`.
- Struktur paling minimal:

```
package main

import "fmt"

func main() {
    fmt.Println("Halo dari Go!")
}
```

- **Beberapa aturan penulisan Go:**
    - Tidak ada `;` di akhir baris

    - Opening brace `{` harus di **akhir baris** yang sama dengan `func`
    - Nama package sesuai folder

- Jalankan program:

```
go run main.go
```

Folder & Package di Go

Slide 3

Aturan inti: satu folder = satu package.

Semua file `.go` di dalam folder yang sama **wajib** punya nama `package` yang sama.

```
myproject/
??? main.go              ? package main
??? utils/
?   ??? string.go        ? package utils
?   ??? number.go        ? package utils
??? internal/
    ??? db/
        ??? conn.go      ? package db
```

Mencampur `package utils` dan `package helper` dalam satu folder akan jadi **error**.

> *Konvensi: nama package sebaiknya sama dengan nama folder (walau secara teknis tidak wajib).*

Folder & Package di Go (2)

Slide 4

Yang menentukan sebuah file masuk package mana adalah **foldernya**, bukan nama yang ditulis di `package ...`.

Begitu masuk subfolder, itu **package terpisah** — walau namanya sama.

```
myproject/
??? main.go              ? package main      (punya folder sendiri)
??? cmd/
?   ??? server/main.go   ? package main      ? package berbeda
?   ??? cli/main.go      ? package main      ? package berbeda
?   ??? worker/main.go   ? package main      ? package berbeda
??? internal/
    ??? db/
        ??? conn.go      ? package db
```

- Kedalaman folder **bebas**, tidak harus rata.
- `package main` bisa ada di root folder, atau dibuat folder sendiri juga boleh.
- Tiap folder `main` = satu executable → `go build ./cmd/server`

Variabel: `var` vs `:=`

Slide 5

- Ada **dua cara** utama untuk deklarasi variabel di Go
- **Cara 1:** `var` — deklarasi eksplisit, bisa tanpa nilai awal

```
var nama string         // default: ""
var umur int            // default: 0
var alamat = "Jakarta"  // tipe otomatis (string)
```

- **Cara 2:** `:=` — short variable declaration, hanya di dalam fungsi

```
nama := "Budi"          // tipe otomatis (string)
umur := 30              // tipe otomatis (int)
```

- Penggunaan
  - `var` — saat deklarasi di level package, atau saat kita butuh nilai default

    - `:=` — lebih ringkas di dalam fungsi (paling sering dipakai)

- Aturan `:=`:
    - **Hanya boleh di dalam fungsi**
    - Minimal satu variabel di sisi kiri harus baru

- Contoh kesalahan umum:

```
nama := "Andi"
nama := "Budi"  // ERROR. Kenapa?
nama = "Budi"   // BENAR
```

Tipe Data Dasar & Zero Value

Slide 6

- Tipe data yang paling sering dipakai:
  - `string` — teks, contoh: `"Halo"`
  - `int` — bilangan bulat, contoh: `42`
  - `float64` — bilangan desimal, contoh: `3.14`
  - `bool` — benar/salah, contoh: `true`
- Tipe tambahan yang sering muncul:
  - `int64`, `int32` — varian int dengan ukuran tertentu
  - `byte` — alias untuk `uint8`
  - `rune` — alias untuk `int32`, untuk karakter Unicode
- Tipe variabel akan otomatis di-set sesuai value:

```
var a = 'A'    // rune (int32)   ? kutip tunggal
var b = "A"    // string          ? kutip ganda
var c = 65     // int             ? angka biasa
```

- **Zero value**, nilai default saat variabel dideklarasi tanpa inisialisasi:
  - `string` → `""` (string kosong)
  - `int` → `0`
  - `float64` → `0.0`
  - `bool` → `false`
  - pointer, slice, map → `nil`

Fungsi di Go

Slide 7

- Fungsi adalah blok kode yang bisa dipanggil berkali-kali

```
func namaFungsi(param1 tipe1, param2 tipe2) tipeReturn {
    // logika
    return nilai
}
```

- Contoh fungsi sederhana:

```
func sapa(nama string) string {
    return "Halo, " + nama + "!"
}

hasil := sapa("Budi")  // "Halo, Budi!"
```

- **Multiple return values**:

```
func bagi(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("pembagi tidak boleh nol")
    }
    return a / b, nil
}

hasil, err := bagi(10, 2)
if err != nil {
    // tangani error
}
```

- Catatan:
  - Tipe parameter ditulis **setelah nama** (kebalikan C++)
  - Bisa return lebih dari satu nilai, misalnya `(result, error)`
  - `func main()` adalah entry point, **tidak** punya parameter & return

Percabangan (Conditional)

Slide 8

```
umur := 20

if umur < 17 {
    fmt.Println("Belum boleh")
} else if umur < 21 {
    fmt.Println("Boleh dengan izin")
} else {
    fmt.Println("Boleh")
}
```

Ciri khas Go:

- Kondisi **tanpa tanda kurung** `( )`
- Kurung kurawal `{ }` **wajib**, walau satu baris

switch — lebih rapi daripada if bertingkat

```
switch metode {
case "GET":
```

```
    fmt.Println("Ambil data")
case "POST":
    fmt.Println("Kirim data")
default:
    fmt.Println("Metode lain")
}
```

Perulangan (Looping)

Slide 9

Golang hanya punya satu keyword loop: `for`

Tidak ada `while`, tidak ada `do-while`. Semua pakai `for`.

a) Bentuk klasik (dengan counter)

```
for i := 0; i < 5; i++ {
    fmt.Println("baris ke-", i)
}
```

b) Seperti `while`

```
i := 0
for i < 5 {
    fmt.Println(i)
    i++
}
```

c) Loop tak terbatas

```
for {
    // jalan terus sampai di-stop
}
```

`for range`, biasa dipakai di web development

Dipakai untuk iterasi slice & map.

```
// Iterasi slice
buah := []string{"apel", "jeruk", "mangga"}
for i, b := range buah {
    fmt.Println(i, b)
}

// Iterasi map
harga := map[string]int{"apel": 5000, "jeruk": 3000}
for nama, hrg := range harga {
    fmt.Println(nama, hrg)
}
```

`struct`: Cetakan Data

Slide 10

- `struct` adalah kumpulan field dengan tipe berbeda
- Seperti "cetakan kue" yang tiap cetakan menghasilkan kue dengan bentuk yang sama
- Definisi struct:

```
type User struct {
    ID    int
    Name  string
    Email string
}
```

- Membuat instance struct:

```
// Cara 1: zero value
var u1 User
u1.Name = "Andi"
u1.Email = "andi@mail.com"
```

```
// Cara 2: struct literal
u2 := User{
    ID:    1,
    Name:  "Budi",
    Email: "budi@mail.com",
}

// Cara 3: positional
u3 := User{2, "Citra", "citra@mail.com"}
```

- Mengakses field:

```
fmt.Println(u2.Name)   // "Budi"
u2.Email = "budi@baru.com"
```

- **Kenapa struct penting di web?**
  - Data dari request, response, dan database biasanya dibungkus dalam struct
  - Contoh: `User`, `Product`, `Order`, `Article`
- Tips: nama field di-export (diawali huruf besar) agar bisa diakses dari package lain

`slice`: Daftar Dinamis

Slide 11

- `slice` adalah list yang ukurannya bisa berubah — padanan "array dinamis" di bahasa lain
- Biasanya, data list di Go disimpan sebagai slice
- Cara membuat slice:

```
// Dari literal
buah := []string{"apel", "jeruk", "mangga"}

// Dari make (ukuran awal 0, kapasitas 5)
angka := make([]int, 0, 5)

// Slice dari slice lain
duaBuah := buah[0:2]  // ["apel", "jeruk"]
```

- Operasi penting:

```
buah = append(buah, "anggur")   // tambah di akhir
fmt.Println(len(buah))          // jumlah elemen: 4
```

- Iterasi dengan `range`:

```
for i, item := range buah {
    fmt.Printf("%d: %s\n", i, item)
}
// atau kalau index tidak perlu:
for _, item := range buah {
    fmt.Println(item)
}
```

- `slice` vs `array`:
    - `array` — ukuran tetap, jarang dipakai langsung
    - `slice` — ukuran dinamis, yang sering dipakai

`map`: Key-Value

Slide 12

- `map` adalah struktur data **key-value** — padanan "dictionary" atau "object"
- Berguna untuk: cache, konfigurasi, lookup cepat
- Cara membuat map:

```
// Dengan tipe eksplisit
harga := map[string]int{
    "apel":  5000,
    "jeruk": 3000,
}

// Dengan make
userCache := make(map[int]string)

// Zero value: nil
var m map[string]int  // nil
```

- Operasi pada map:

```
harga["mangga"] = 7000        // tambah/update
delete(harga, "jeruk")        // hapus
nilai := harga["apel"]        // baca (0 jika tidak ada)
fmt.Println(len(harga))       // jumlah entry
```

- **Pola `value, ok` — cara aman membaca map:**

```
nilai, ada := harga["durian"]
if !ada {
    fmt.Println("durian tidak ada di map")
}
```

- Iterasi map (urutan akan random):

```
for key, val := range harga {
    fmt.Printf("%s: %d\n", key, val)
}
```

Pointer Sekilas: `*T` dan `&`

Slide 13

- Pointer adalah variabel yang menyimpan **alamat memori** dari variabel lain
- Di Go: `*T` artinya pointer ke tipe `T`, `&x` artinya alamat memori `x`
- Kenapa pointer ada di Go?
  - **Performance**, tidak perlu copy data besar
  - **Modifikasi data** lewat parameter fungsi
- Contoh dasar:

```
x := 42
p := &x           // p adalah pointer ke x
fmt.Println(*p)   // 42 (dereference, baca nilai)
*p = 100          // ubah nilai x lewat pointer
fmt.Println(x)    // 100
```

- Kapan kita pakai pointer di web?

- Saat kita ingin fungsi mengubah variabel di luar scope
- Saat pakai struct besar agar tidak di-copy

Error Handling Eksplisit

Slide 14

- Go **tidak** punya `try/catch`
- Error diperlakukan sebagai **nilai biasa** yang dikembalikan dari fungsi
- Pola standar:

```
result, err := someFunction()
if err != nil {
    // tangani error
    return  // atau fallback
}
// pakai result
```

- Contoh nyata: membuka file

```
data, err := os.ReadFile("config.json")
if err != nil {
    log.Fatal("Gagal baca file:", err)
}
```

```
fmt.Println(string(data))
```

- Membuat error sendiri:

```
import "errors"

func bagi(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("pembagi tidak boleh nol")
    }
    return a / b, nil
}
```

- Kadang pakai `fmt.Errorf` untuk error berformat:

```
return 0, fmt.Errorf("user %d tidak ditemukan", id)
```

- **Kenapa eksplisit?**
  - Kita **dipaksa** memikirkan error di setiap langkah
  - Tidak ada error yang "tidak tertangkap" diam-diam

- Kode lebih panjang, tapi alur error jelas terlihat

Tugas Minggu Depan

- **Tugas individu (dikumpulkan pertemuan depan):**
  - Buat program "Daftar Belanja" dengan:
    - `struct Item` (Nama, Harga, Jumlah)
    - Slice untuk menyimpan item
    - Fungsi untuk tambah, hitung total, tampilkan
    - Validasi: harga dan jumlah harus > 0
  - Submit: file `.go` + screenshot output dalam zip